

From Computational Graphs to Contextual Routing: A Comprehensive Analysis of Optimization and Transformer Architectures

Sahki Abderrahemane

AI Engineer & Deep Learning Enthusiast

June 2026

Contents

1	Introduction	2
2	Advanced Optimization & Computational Graphs	2
2.1	Backpropagation via Jacobians	2
2.2	The Loss Landscape	3
2.3	Classical Optimizers: Momentum, RMSProp, and AdaGrad	4
2.3.1	SGD with Momentum	4
2.3.2	AdaGrad	5
2.3.3	RMSProp	5
2.4	Adam & AdamW: The Modern Standard	6
2.4.1	Adam	6
2.4.2	AdamW: Decoupled Weight Decay	6
3	Regularization & Training Dynamics	7
3.1	Normalization: Batch & Layer	7
3.1.1	Batch Normalization (BN)	7
3.1.2	Layer Normalization (LN)	8
3.2	Mixed Precision Training (FP16/BF16)	8
3.3	Gradient Clipping & Gradient Accumulation	9
4	The Self-Attention Mechanism & Sequence Grounding	10
4.1	Scaled Dot-Product Attention	10
4.2	Multi-Head Attention	11
4.3	Positional Encodings	12
4.3.1	Sinusoidal Positional Encoding	12
4.3.2	Rotary Position Embedding (RoPE)	12
5	Transformer Architectural Routing Paradigms	13
5.1	Graph Routing Intuition	13
5.2	BERT: Bidirectional Encoder	13
5.3	GPT: Causal Decoder	14
5.4	T5: Encoder-Decoder with Cross-Attention	14
5.5	Visualizing Attention Masks	15
5.6	Architecture Overview	15
6	Conclusion	15

1 Introduction

Deep learning has evolved from simple layered perceptrons to massive attention-based architectures that dominate natural language processing and beyond. Two fundamental pillars support this progress: robust optimization theory, which provides the mathematical machinery to train deep models, and architectural innovations, which structure how information flows through a network. This report offers a self-contained journey through both pillars.

We begin by dissecting computational graphs and the Jacobian-based backpropagation that makes gradient computation automatic and efficient. We then explore the geometry of the loss landscape, contrasting sharp and flat minima, and detail the most impactful optimizers: from classical SGD with momentum and RMSProp to the modern Adam and AdamW, including the crucial distinction between L_2 regularization and decoupled weight decay. Normalization techniques—Batch and Layer Normalization—are examined with a mathematical lens, explaining why LayerNorm is preferred for sequential data. We also discuss practical training strategies: mixed precision with loss scaling, gradient clipping, and gradient accumulation.

The second half of the report transitions to the Transformer architecture. We derive the scaled dot-product attention, motivate positional encodings, and then analyze three routing paradigms: the open mask of BERT, the causal mask of GPT, and the encoder-decoder cross-attention of T5. Throughout, rigorous mathematics is paired with PyTorch implementations and visual diagrams to build intuition. By the end, the reader will possess a deep understanding of the optimization and architectural principles that power modern deep learning.

2 Advanced Optimization & Computational Graphs

2.1 Backpropagation via Jacobians

A neural network can be viewed as a directed acyclic computational graph $G = (V, E)$. Each node performs an operation, and edges carry tensors. The forward pass computes a loss L by composing functions. To obtain gradients, backpropagation traverses the graph in reverse topological order using the chain rule. For a node computing $\mathbf{z} = f(\mathbf{x}, \mathbf{y})$, the Jacobians $\partial \mathbf{z} / \partial \mathbf{x}$ and $\partial \mathbf{z} / \partial \mathbf{y}$ are multiplied by the upstream gradient $\partial L / \partial \mathbf{z}$ to produce downstream gradients.

PyTorch Implementation: Custom Autograd Function Listing 1 shows

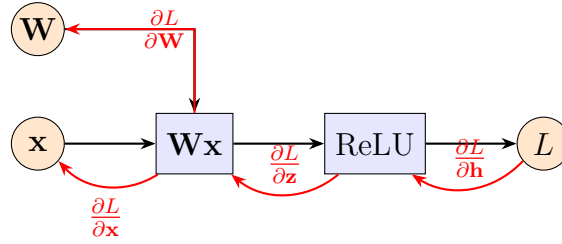


Figure 1: Forward computation (black) and backward gradient flow (red). Local Jacobians are chained to compute gradients with respect to all parameters.

how PyTorch implements the Jacobian-vector product inside a custom ‘autograd.Function’.

```

1 import torch
2 from torch.autograd import Function
3
4 class LinearFunction(Function):
5     @staticmethod
6     def forward(ctx, input, weight, bias=None):
7         ctx.save_for_backward(input, weight)
8         output = input.matmul(weight.t())
9         if bias is not None:
10             output += bias
11         return output
12
13     @staticmethod
14     def backward(ctx, grad_output):
15         input, weight = ctx.saved_tensors
16         grad_input = grad_output.matmul(weight)
17         grad_weight = grad_output.t().matmul(input)
18         return grad_input, grad_weight, None

```

Listing 1: Custom linear layer illustrating Jacobian computation in backpropagation.

2.2 The Loss Landscape

The loss function $L(\theta)$ of a deep network defines a high-dimensional surface. Critical points where $\nabla L(\theta) = \mathbf{0}$ can be minima, maxima, or saddle points. The local curvature is captured by the Hessian matrix $\mathbf{H} = \nabla^2 L(\theta)$. In high dimensions, saddle points—where some eigenvalues are positive and others negative—are far more common than true minima. Efficient optimizers need to escape such points rapidly.

A central theme in generalization is the preference for **flat minima** over sharp ones. Flat minima have smaller eigenvalues of the Hessian, meaning that small perturbations of the parameters do not drastically increase the loss. This robustness often correlates with better test performance.

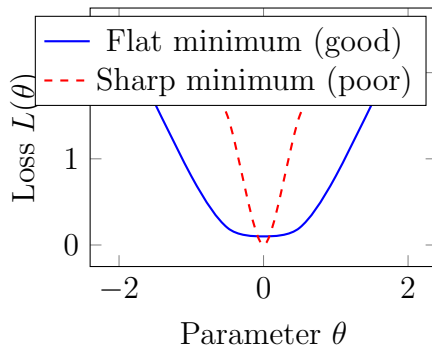


Figure 2: Illustration of flat vs. sharp minima in 1D. Flat minima are more robust to parameter perturbations, leading to better generalization.

2.3 Classical Optimizers: Momentum, RMSProp, and AdaGrad

Before adaptive methods like Adam, several foundational optimizers addressed the limitations of vanilla SGD. We review them here because their ideas are reused in modern optimizers.

2.3.1 SGD with Momentum

Momentum accumulates a velocity vector $\mathbf{v}_t$ that helps the optimizer push through narrow valleys and small oscillations:

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla L(\boldsymbol{\theta}_t), \quad (1)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t. \quad (2)$$

$\gamma \in [0, 1)$ is the momentum coefficient (often 0.9). The update direction is a smoothed average of past gradients.

2.3.2 AdaGrad

AdaGrad adapts the learning rate per parameter by dividing by the square root of the sum of squared historical gradients:

$$s_t = s_{t-1} + (\nabla L(\boldsymbol{\theta}_t))^2, \quad (3)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \nabla L(\boldsymbol{\theta}_t). \quad (4)$$

It performs well for sparse features but accumulates the squared gradients indefinitely, causing the learning rate to decay to zero prematurely.

2.3.3 RMSProp

RMSProp fixes AdaGrad’s monotonically decreasing learning rate by using a moving average of squared gradients, similarly to how Adam later employs it:

$$s_t = \beta s_{t-1} + (1 - \beta)(\nabla L(\boldsymbol{\theta}_t))^2, \quad (5)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \nabla L(\boldsymbol{\theta}_t). \quad (6)$$

Here β controls the decay rate (commonly 0.9). This allows the effective learning rate to adapt to recent gradient magnitudes without decaying to zero. RMSProp is particularly popular for recurrent networks.

PyTorch Implementation: RMSProp from Scratch Listing 2 shows a simple manual implementation that mirrors the internal optimizer.

```
1 def rmsprop_step(params, sq_avgs, lr=1e-3, beta=0.9, eps=1e-8):
2     with torch.no_grad():
3         for p, s in zip(params, sq_avgs):
4             grad = p.grad
5             if grad is None:
6                 continue
7             s.mul_(beta).add_((1 - beta) * grad ** 2)
8             p.sub_(lr * grad / (s.sqrt() + eps))
```

Listing 2: Manual RMSProp update step.

2.4 Adam & AdamW: The Modern Standard

2.4.1 Adam

Adam combines the momentum (first moment) and RMSProp (second moment) with bias correction:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \quad (8)$$

The bias-corrected estimates are $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$, and the update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (9)$$

Adam adapts the effective learning rate per parameter and includes momentum, making it robust to many problems.

2.4.2 AdamW: Decoupled Weight Decay

Standard L_2 regularization adds $\frac{\lambda}{2} \|\theta\|^2$ to the loss, modifying the gradient to $g_t + \lambda \theta$. In adaptive optimizers like Adam, this addition interacts with the second-moment estimate, causing sub-optimal regularization. AdamW decouples weight decay by applying it directly:

$$\theta_{t+1} = (1 - \eta\lambda) \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (10)$$

This simple change yields better generalization and is the default for training Transformers.

PyTorch Implementation: AdamW update logic Listing 3 demonstrates the core of the AdamW update step (excluding bias correction for brevity).

```
1 def adamw_step(params, optimizer_state, lr, betas,
2   weight_decay, eps):
3     beta1, beta2 = betas
4     for p, state in zip(params, optimizer_state):
5         grad = p.grad
6         if grad is None:
7             continue
8         state['step'] += 1
9         state['m'] = beta1 * state['m'] + (1 - beta1) * grad
10        state['v'] = beta2 * state['v'] + (1 - beta2) * grad
11        **2
12        m_hat = state['m'] / (1 - beta1 ** state['step'])
```

```

11     v_hat = state['v'] / (1 - beta2 ** state['step'])
12     p.mul_(1 - lr * weight_decay)
13     p.sub_(lr * m_hat / (v_hat.sqrt() + eps))

```

Listing 3: Core logic of the AdamW optimizer.

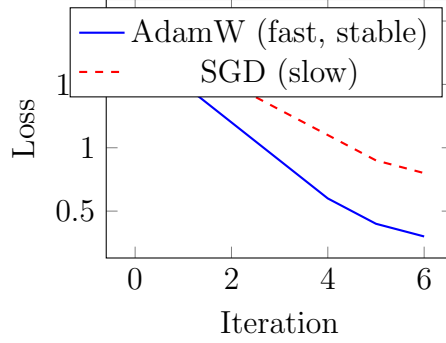


Figure 3: Typical convergence behavior: Adam(W) adapts per-parameter learning rates and often converges faster and to a better minimum than plain SGD.

3 Regularization & Training Dynamics

3.1 Normalization: Batch & Layer

Normalization layers stabilize training by reducing internal covariate shift and smoothing the optimization landscape.

3.1.1 Batch Normalization (BN)

For a mini-batch of size m and feature dimension d , BN normalizes across the batch for each feature independently:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_{ij}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_{ij} - \mu_j)^2, \quad (11)$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j. \quad (12)$$

The statistics are computed over the batch, so BN behavior depends on batch size and is inconsistent between training and inference (running averages are used for inference).

3.1.2 Layer Normalization (LN)

Layer Normalization transposes the normalization axis: for a single example with d features, it computes statistics over the feature dimension:

$$\mu_i = \frac{1}{d} \sum_{j=1}^d x_{ij}, \quad \sigma_i^2 = \frac{1}{d} \sum_{j=1}^d (x_{ij} - \mu_i)^2, \quad (13)$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \quad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j. \quad (14)$$

LayerNorm is independent of batch size and works identically at training and inference, making it ideal for variable-length sequences in Transformers.

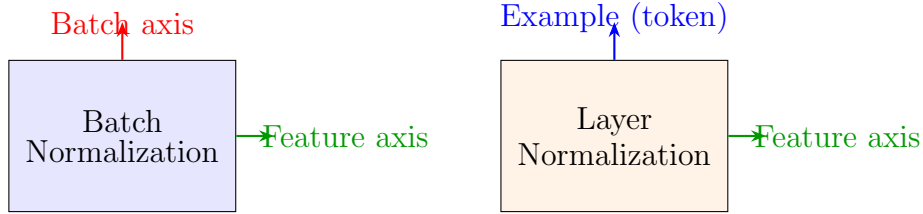


Figure 4: BatchNorm normalizes over the batch dimension for each feature; LayerNorm normalizes over the feature dimension for each example. This makes LayerNorm suitable for sequences with variable lengths.

LayerNorm Implementation Listing 4 shows a simplified forward pass of LayerNorm written from scratch.

```

1 import torch
2
3 def layer_norm_forward(x, gamma, beta, eps=1e-5):
4     # x: (batch, features)
5     mean = x.mean(dim=-1, keepdim=True)
6     var = x.var(dim=-1, keepdim=True, unbiased=False)
7     x_norm = (x - mean) / torch.sqrt(var + eps)
8     return gamma * x_norm + beta

```

Listing 4: Simplified LayerNorm forward pass.

3.2 Mixed Precision Training (FP16/BF16)

Training in mixed precision stores weights in FP32 but computes forward/backward in lower precision (FP16 or BF16) for speed and memory savings. The

main numerical danger is gradient underflow: small gradients may become zero. A loss scale factor S shifts the gradients:

$$\text{scaled_loss} = S \cdot L, \quad \text{gradients scaled by } S. \quad (15)$$

After backpropagation, gradients are divided by S before the optimizer step. **PyTorch Mixed Precision Example** Listing 5 shows a training step with automatic mixed precision using ‘torch.cuda.amp’.

```

1 scaler = torch.cuda.amp.GradScaler()
2 for data, target in dataloader:
3     optimizer.zero_grad()
4     with torch.cuda.amp.autocast():
5         output = model(data)
6         loss = loss_fn(output, target)
7     scaler.scale(loss).backward()
8     scaler.step(optimizer)
9     scaler.update()

```

Listing 5: Mixed precision training with gradient scaling.

3.3 Gradient Clipping & Gradient Accumulation

Gradient clipping prevents exploding gradients by scaling the gradient vector when its norm exceeds a threshold:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g}, & \|\mathbf{g}\|_2 \leq \tau, \\ \tau \frac{\mathbf{g}}{\|\mathbf{g}\|_2}, & \|\mathbf{g}\|_2 > \tau. \end{cases} \quad (16)$$

Gradient accumulation simulates larger batch sizes by summing gradients over several micro-batches before updating weights.

Implementation Snippets

```

1 # Clipping
2 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm
   =1.0)
3
4 # Accumulation (manual)
5 accumulation_steps = 4
6 for i, (data, target) in enumerate(dataloader):
7     loss = model(data)
8     loss.backward()
9     if (i + 1) % accumulation_steps == 0:
10         optimizer.step()
11         optimizer.zero_grad()

```

Listing 6: Gradient clipping and accumulation.

4 The Self-Attention Mechanism & Sequence Grounding

4.1 Scaled Dot-Product Attention

Given queries $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, and values $\mathbf{V} \in \mathbb{R}^{T \times d_v}$, the attention output is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (17)$$

The dot product $\mathbf{Q}\mathbf{K}^\top$ can grow large as d_k increases, pushing the softmax into regions of near-zero gradient. Dividing by $\sqrt{d_k}$ keeps the variance of the logits constant (assuming uncorrelated entries with variance $1/d_k$), maintaining healthy gradient flow.

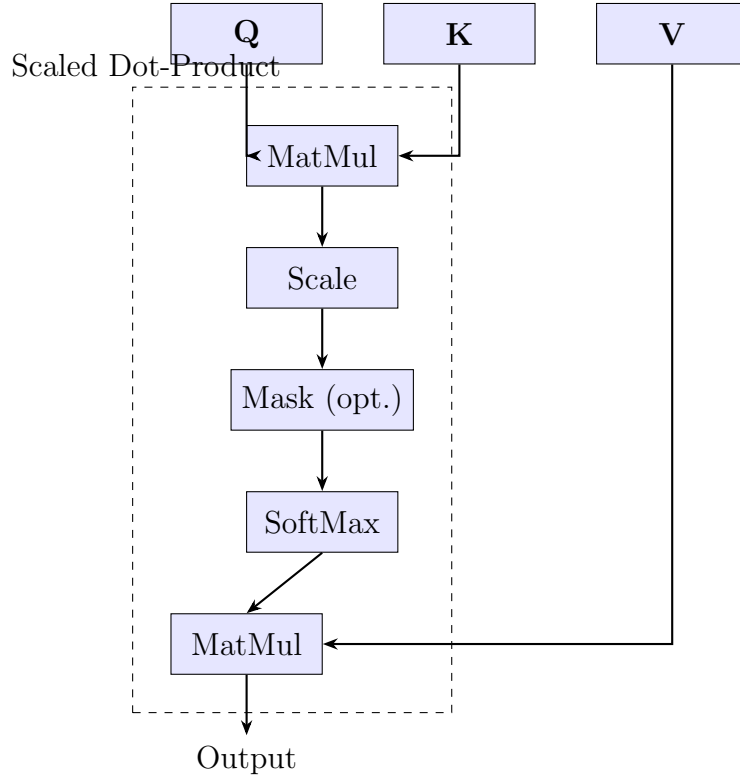


Figure 5: Block diagram of scaled dot-product attention. The mask (optional) is added before softmax.

4.2 Multi-Head Attention

Instead of a single attention function, multiple heads are used in parallel, each with its own set of projections, to capture different representation subspaces. The outputs are concatenated and linearly projected:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (18)$$

where $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

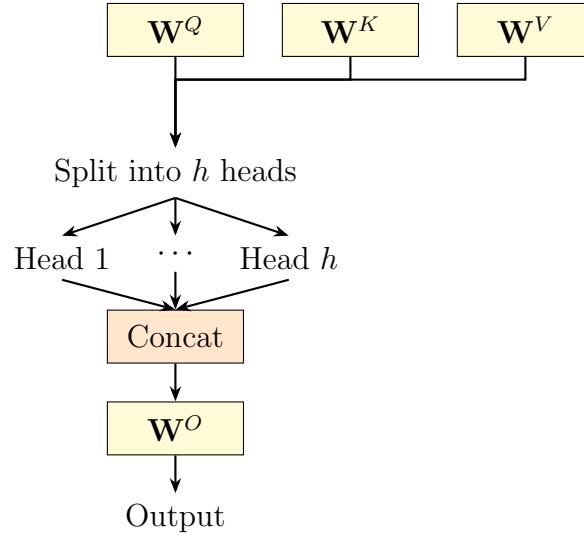


Figure 6: Multi-head attention: queries, keys, values are projected and split into h heads, processed in parallel, concatenated, and projected.

PyTorch Implementation: Full Multi-Head Attention Listing 7 shows a complete multi-head attention module.

```

1 import torch.nn as nn
2 import torch.nn.functional as F
3
4 class MultiHeadAttention(nn.Module):
5     def __init__(self, d_model, n_heads, dropout=0.1):
6         super().__init__()
7         assert d_model % n_heads == 0
8         self.d_k = d_model // n_heads
9         self.n_heads = n_heads
10        self.q_linear = nn.Linear(d_model, d_model)
11        self.k_linear = nn.Linear(d_model, d_model)
12        self.v_linear = nn.Linear(d_model, d_model)
13        self.out_linear = nn.Linear(d_model, d_model)
14        self.dropout = nn.Dropout(dropout)

```

```

15
16 def forward(self, Q, K, V, mask=None):
17     bs = Q.size(0)
18     Q = self.q_linear(Q).view(bs, -1, self.n_heads, self.
19         d_k).transpose(1,2)
20     K = self.k_linear(K).view(bs, -1, self.n_heads, self.
21         d_k).transpose(1,2)
22     V = self.v_linear(V).view(bs, -1, self.n_heads, self.
23         d_k).transpose(1,2)
24     attn_scores = torch.matmul(Q, K.transpose(-2,-1)) / (
25         self.d_k ** 0.5)
26     if mask is not None:
27         attn_scores = attn_scores + mask
28         attn_weights = F.softmax(attn_scores, dim=-1)
29         attn_weights = self.dropout(attn_weights)
30         attn_output = torch.matmul(attn_weights, V)
31         attn_output = attn_output.transpose(1,2).contiguous()
32         .view(bs, -1, self.n_heads * self.d_k)
33     return self.out_linear(attn_output)

```

Listing 7: Multi-head attention with optional mask.

4.3 Positional Encodings

Self-attention is permutation-invariant. To inject order, positional encodings are added to token embeddings.

4.3.1 Sinusoidal Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (19)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (20)$$

They are deterministic and allow the model to easily learn relative distances.

4.3.2 Rotary Position Embedding (RoPE)

RoPE applies a rotation to query and key vectors. For a pair (x_{2i}, x_{2i+1}) at position m , the rotation angle is $\theta_i = 10000^{-2i/d}$. The rotated vector is

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}. \quad (21)$$

The dot product $\mathbf{q}_m \cdot \mathbf{k}_n$ after RoPE depends only on the relative position $m - n$, enabling better length generalization.

Generating Sinusoidal Encodings

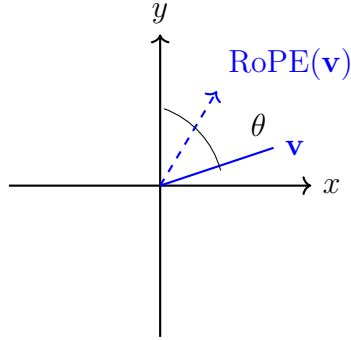


Figure 7: RoPE rotates each 2D pair of query/key dimensions by a position-dependent angle.

```

1 def sinusoidal_encoding(max_len, d_model):
2     pe = torch.zeros(max_len, d_model)
3     position = torch.arange(0, max_len).unsqueeze(1).float()
4     div_term = torch.exp(torch.arange(0, d_model, 2).float()
5         *
6         -(torch.log(torch.tensor(10000.0)) /
7           d_model))
8     pe[:, 0::2] = torch.sin(position * div_term)
9     pe[:, 1::2] = torch.cos(position * div_term)
10    return pe

```

Listing 8: Sinusoidal positional encoding.

5 Transformer Architectural Routing Paradigms

5.1 Graph Routing Intuition

A Transformer can be interpreted as a fully connected graph on T token nodes. The attention weights $\mathbf{A} = \text{softmax}(\mathbf{QK}^\top / \sqrt{d_k} + \mathbf{M})$ define weighted edges. An additive mask $\mathbf{M}$ sets forbidden edges to $-\infty$, zeroing them after softmax. This hard-wires the information flow.

5.2 BERT: Bidirectional Encoder

BERT uses an encoder-only Transformer with a mask $\mathbf{M} = \mathbf{0}$. Every token attends to every other token, left and right. This bidirectional context is ideal for classification and understanding tasks.

5.3 GPT: Causal Decoder

GPT models generate left-to-right. The causal mask $\mathbf{M}$ is lower-triangular with $-\infty$ in the upper triangle:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (22)$$

Because $\lim_{x \rightarrow -\infty} e^x = 0$, the softmax weights for future tokens become exactly zero. This prevents information leakage.

Causal Mask Implementation

```
1 import torch
2
3 def causal_mask(seq_len, device=None):
4     mask = torch.triu(torch.ones(seq_len, seq_len, device=
5         device), diagonal=1)
6     mask = mask.masked_fill(mask == 1, float('-inf'))
7     return mask
```

Listing 9: Causal mask generation.

5.4 T5: Encoder-Decoder with Cross-Attention

T5’s encoder processes the input bidirectionally (open mask). The decoder generates autoregressively with causal self-attention and also uses **cross-attention**, where queries come from the decoder and keys/values from the encoder output (bidirectional mask). This enables tasks like translation and summarization.

Simplified T5-Style Cross-Attention Forward

```
1 class CrossAttention(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         self.attn = MultiHeadAttention(d_model, n_heads)
5
6     def forward(self, decoder_hidden, encoder_output,
7         decoder_mask=None):
8         return self.attn(decoder_hidden, encoder_output,
9             encoder_output, mask=None)
```

Listing 10: Cross-attention layer between encoder and decoder.

5.5 Visualizing Attention Masks

The mask matrix directly encodes the allowed connections. Figure 8 contrasts the open mask (BERT) and causal mask (GPT). For a sequence length $T = 5$, the open mask is all zeros, while the causal mask has $-\infty$ (shown in red) in the upper triangle.

BERT Mask (Open)					GPT Mask (Causal)				
0	0	0	0	0	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
0	0	0	0	0	0	0	$-\infty$	$-\infty$	$-\infty$
0	0	0	0	0	0	0	0	$-\infty$	$-\infty$
0	0	0	0	0	0	0	0	0	$-\infty$
0	0	0	0	0	0	0	0	0	0

Figure 8: Attention mask matrices for $T = 5$. Left: BERT’s open mask (all zeros). Right: GPT’s causal mask (lower-triangular zeros, upper-triangular $-\infty$). Red cells are prohibited connections.

5.6 Architecture Overview

Figure 9 summarizes the three main Transformer paradigms: encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5). The flow of information and the type of attention used are indicated.

6 Conclusion

This report has traversed the deep learning stack from gradient computation to specialized Transformer routing. We dissected backpropagation via Jacobians, explored the loss landscape, and detailed classical and modern optimizers—including RMSProp, Adam, and AdamW—with precise mathematics and PyTorch code. Normalization techniques (LayerNorm) were shown to be critical for stable training, and practical tricks (mixed precision, gradient clipping, accumulation) were demystified.

On the architectural side, self-attention with positional encodings enables powerful sequence modeling. The routing masks of BERT, GPT, and T5 define distinct information flow paradigms, each suited to different tasks. These components, combined with robust optimization, form the backbone of

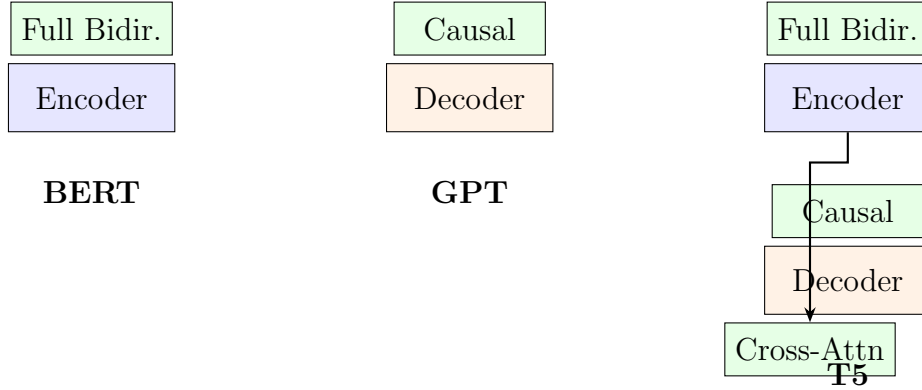


Figure 9: High-level architectures. BERT uses only an encoder with full bidirectional attention. GPT uses a decoder with causal self-attention. T5 pairs an encoder (bidirectional) with a decoder that has causal self-attention and cross-attention to the encoder output.

today’s state-of-the-art language models. Understanding their interplay is essential for any deep learning practitioner aiming to build or adapt large-scale systems.

References

- [1] A. Vaswani *et al.*, “Attention is all you need,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [2] J. Devlin *et al.*, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [3] A. Radford *et al.*, “Improving language understanding by generative pre-training,” OpenAI, 2018.
- [4] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Int. Conf. Learn. Represent.*, 2019.

- [7] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [8] T. Tieleman and G. Hinton, “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,” *COURSERA: Neural Networks for Machine Learning*, 2012.
- [9] P. Micikevicius *et al.*, “Mixed precision training,” in *Int. Conf. Learn. Represent.*, 2018.
- [10] J. Su *et al.*, “RoFormer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021.